

BIT4107 — Mobile Application Development

Student Logbook: Weeks 1 – 8

Student Name	Makwae Ethan Hope
Registration Number	BIT/2023/72445
University	Mount Kenya University
School / Faculty	School of Computing and Informatics
Programme	Bachelor of Science in Information Technology
Unit Code	BIT4107
Unit Name	Mobile Application Development
Lecturer	Michael Nyoro

Table of Contents

1. [Week 1 — Introduction to Mobile Development](#)
2. [Week 2 — Mobile Development Environment & Programming Frameworks](#)
3. [Week 3 — User Interface Design](#)
4. [Week 4 — Event Handling and Data Management](#)
5. [Week 5 — Networking and APIs](#)
6. [Week 6 — CAT 1: Integrated Application Assessment](#)
7. [Week 7 — Advanced Database Design and Security](#)
8. [Week 8 — Handling User Input and Touch Gestures](#)

Summary Table

Week	Topic	Key Activities	Skills Gained	Challenges	Outcome
1	Introduction to Mobile Dev	Installed Flutter SDK, Dart, VS Code, configured ADB, created first project	Flutter setup, project structure, device connection	Configuring system PATH for Flutter, enabling USB debugging on TECNO BF7	Successfully ran Hello World app on physical device
2	Programming & Frameworks	Built login and registration screens, implemented SharedPreferences session management	Forms, TextFormField, Navigator, SharedPreferences	Managing login state persistence across sessions	Functional login/register flow with persistent session
3	UI/UX Design	Designed Dashboard, Profile, Welcome, Settings screens with dark mode	Material Design, StatefulWidget, dark mode, multi-screen navigation	Routing between many screens consistently	Complete multi-screen app with blue (#1565C0) colour scheme
4	Event Handling & Data Management	Implemented SQLite for Notes (full CRUD) and Timetable (full CRUD + search)	SQLite, DatabaseHelper, form validation, real-time search	Schema design, version management, query ordering	Persistent data storage with search and edit capabilities
5	Networking & APIs	Integrated DEV.to REST API, JSON parsing, 4-type error handling, pull-to-refresh	HTTP GET, async/await, JSON parsing, error handling	Handling all network failure states without crashing	Live news feed from public API with robust error handling
6	CAT 1 Assessment	Built Reports screen aggregating data from all sources, wrote Technical Report	SQL COUNT queries, GROUP BY aggregation, data aggregation	Combining multiple async data sources in one screen	Comprehensive reports screen covering all app data sources
7	Database Design & Security	SHA-256 password hashing, attendance SQLite table (DB v3), attendance CRUD UI	Cryptographic hashing, database migration, 3NF design	Password migration for existing users, attendance schema	Secure password storage; full attendance tracking module
8	User Input & Gestures	OOP controller classes, GestureDetector, keyboard input, live validation, event log	GestureDetector, FocusNode, OOP design, event-driven programming	Detecting swipe direction from velocity, live validation feedback	Interactive gestures demo with real-time event log

Week 1 — Introduction to Mobile Development

1.1 Overview

This week marked the beginning of the BIT4107 Mobile Application Development unit. The primary focus was understanding the mobile development ecosystem, installing the necessary tools, and successfully running a first Flutter application on a physical Android device.

1.2 Tools and Software Used

Tool	Version / Details	Purpose
Flutter SDK	3.x (stable channel)	Cross-platform mobile development framework
Dart SDK	Bundled with Flutter	Programming language for Flutter
Visual Studio Code	Latest	Primary code editor
Flutter Extension	VS Code plugin	Syntax highlighting, hot reload, debugging
Dart Extension	VS Code plugin	Dart language support
Android Debug Bridge (ADB)	Platform Tools	Connecting Android device for testing
TECNO BF7	Android device	Physical test device

1.3 Activities Undertaken

Environment Setup: - Downloaded and extracted the Flutter SDK from flutter.dev - Configured the system PATH environment variable to include the Flutter bin directory - Ran `flutter doctor` to verify the development environment — confirmed Flutter, Dart, and Android toolchain as working; noted Visual Studio was not installed (not required for Android development) - Installed VS Code Flutter and Dart extensions from the marketplace

First Project: - Created the Smartz Student Portal project using `flutter create hello_world_app` - Explored the generated project structure:

```
hello_world_app/
├── lib/
│   └── main.dart
├── android/
│   └── Android-specific configuration
├── ios/
│   └── iOS-specific configuration
├── pubspec.yaml
├── Dependencies and project metadata
└── test/
    └── Unit tests
```

- Examined `main.dart` and understood the `MaterialApp`, `Scaffold`, and `StatefulWidget` structure - Enabled USB debugging on TECNO BF7 (Settings → About Phone → tap Build Number 7 times → Developer Options → USB Debugging) - Ran the app using `flutter run` and observed the default counter app on the physical device - Tested hot reload: modified the title string in `main.dart`, saved the file, and observed the change apply instantly without a full rebuild

1.4 Skills Gained

- Understanding of Flutter's widget-based architecture (everything is a widget)
- Experience configuring a mobile development environment from scratch on Windows
- Familiarity with the `pubspec.yaml` dependency file format
- Basic understanding of the Dart programming language (syntax, entry point, classes)
- Connecting a physical Android device for live testing using USB debugging
-

1.5 Challenges and Solutions

Challenge	Solution
flutter command not recognised after installation	Manually added C:\flutter\bin to the Windows system PATH via Environment Variables settings
USB debugging option not visible on TECNO BF7	Found Build Number in Settings → About Phone → Software Information; tapped 7 times to unlock Developer Options
flutter doctor showed Android licenses not accepted	Ran flutter doctor --android-licenses and accepted all licenses

1.6 Personal Reflection

Setting up the development environment took more effort than expected, particularly configuring the system PATH and enabling USB debugging on the test device. However, once `flutter doctor` reported all required tools as working and the first app appeared on the phone screen, the effort felt worthwhile. The hot reload feature was impressive — changes appeared on the device in under a second, which makes iterative design much faster than traditional Android development with Gradle builds. This week established a solid foundation for everything that followed.

Week 2 — Mobile Development Environment & Programming Frameworks

2.1 Overview

This week focused on applying Flutter programming concepts to build the authentication layer of the Smartz Student Portal. The registration and login screens were created with form validation and persistent storage using the `shared_preferences` package.

2.2 Technologies Introduced

Technology	Package	Purpose
SharedPreferences	<code>shared_preferences: ^2.2.2</code>	Persistent key-value local storage
Navigator	Flutter built-in	Screen navigation and routing
TextFormField	Flutter built-in	Validated text input fields
<code>GlobalKey<FormState></code>	Flutter built-in	Form state management and validation

2.3 Activities Undertaken

Registration Screen (register_page.dart): - Built a registration form with fields: Full Name, Email, Registration Number, Password - Added a GlobalKey<FormState> to manage form validation - Implemented validators for each field: - Name: must not be empty - Email: must contain @ symbol - Password: minimum 6 characters - Used TextEditingController to read form values on submission

LocalStorage Class (local_storage.dart): Created a static utility class to wrap all SharedPreferences operations:

```
class LocalStorage {
  static Future<void> saveStudent({
    required String name,
    required String email,
    required String regNumber,
    required String password,
  }) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('student_name', name);
    await prefs.setString('student_email', email);
    await prefs.setString('student_reg', regNumber);
    await prefs.setString('student_password', password);
  }

  static Future<bool> login(String email, String password) async {
    final prefs = await SharedPreferences.getInstance();
    final savedEmail = prefs.getString('student_email');
    final savedPassword = prefs.getString('student_password');
    if (savedEmail == email && savedPassword == password) {
      await prefs.setBool('is_logged_in', true);
      return true;
    }
    return false;
  }
}
```

Login Screen (login_page.dart): - Called LocalStorage.login() on form submission - On success: navigated to Dashboard using Navigator.pushReplacement (prevents going back to login) - On failure: displayed an inline error message using setState

Session Management: - On app launch (main.dart), checked LocalStorage.isLoggedIn() to decide whether to show the Welcome screen or jump directly to the Dashboard

Added dependency to pubspec.yaml:

```
dependencies:
  shared_preferences: ^2.2.2
```

Ran flutter pub get to download the package.

2.4 Skills Gained

- Using `SharedPreferences` for persistent key-value storage (strings, booleans)
- Building validated forms with `Form`, `TextFormField`, and `FormState`
- Navigation patterns: `Navigator.push` vs `Navigator.pushReplacement` and when to use each
- Using `async/await` for asynchronous `SharedPreferences` calls
- Organising code into static utility classes (early OOP practice)

2.5 Challenges and Solutions

Challenge	Solution
App returned to Login screen on hot restart (session lost)	Added <code>is_logged_in</code> boolean flag in <code>SharedPreferences</code> and checked it in <code>main.dart</code> on startup
Form submitted before all fields were validated	Used <code>if (!_formKey.currentState!.validate()) return;</code> before processing form data
Password field showed typed characters	Added <code>obscureText: true</code> property to the password <code>TextFormField</code>

2.6 Personal Reflection

This week showed how even simple local storage requires careful state management. The challenge of keeping a user logged in across app restarts taught me how mobile applications differ from websites — there is no automatic session cookie. Building the `LocalStorage` class as a clean wrapper around `SharedPreferences` was my first real application of encapsulation: the rest of the app does not need to know how data is stored, only that it can call `saveStudent()` and `login()`.

Week 3 — User Interface Design

Date: May – June 2026

3.1 Overview

This week concentrated on building a complete, visually consistent multi-screen user interface for the Smartz Student Portal. A Dashboard with quick-action cards, a Profile screen, a Settings screen with dark mode, and a Welcome screen were all designed and connected.

3.2 Design Standards

Element	Decision
Primary colour	#1565C0 (Material Blue 800)
Background colour	#BBDEFB (Material Blue 50)
Typography	Flutter default (Roboto)
Widget style	Material Design 3
Layout	Column, Row, Expanded, SingleChildScrollView

3.3 Activities Undertaken

Welcome Screen (welcome_page.dart): - Designed a landing screen with the Smartz App logo, a welcome message, and two buttons: Login and Register - Used `Navigator.push` to reach either screen

Dashboard (dashboard_page.dart): - Built a header card showing the logged-in student's name and registration number (read from `SharedPreferences` on `initState`) - Created a 2-column quick-action card grid using `Row` and `Expanded` widgets with `GestureDetector` for tap handling - Cards added: My Profile, My Courses, Assignments, Results, My Notes, My Timetable, Campus News Feed, Reports - Used `automaticallyImplyLeading: false` on the `AppBar` to prevent accidental back-navigation to Login

Profile Screen (profile_page.dart): - Displayed student name, email, registration number, and course name - Included a `CircleAvatar` for the profile photo with `image_picker` integration (pick from gallery)

Settings Screen (settings_page.dart): - Dark mode toggle using `Switch` widget — saved preference to `SharedPreferences` via `LocalStorage.saveTheme()` - Change password form (current password, new password, confirm) - Profile image picker - Logout button that clears the session and routes to Login

Dark Mode Implementation: - In `main.dart`, used `FutureBuilder` to read the theme preference before first render: `dart theme: ThemeData.light(), darkTheme: ThemeData.dark(), themeMode: isDark ? ThemeMode.dark : ThemeMode.light,`

Splash / Route Decider: - Added a `SplashDecider` widget that checks `isLoggedIn()` asynchronously and routes to either the Welcome screen or the Dashboard — so returning users skip the welcome flow entirely.

3.4 Skills Gained

- Applying Material Design principles to create a consistent look and feel
- Building reusable widget methods (`_buildQuickAction`, `_buildInfoCard`)
- Managing app-wide state (dark mode) through `SharedPreferences`
- Using `Navigator.pushReplacement` for non-reversible navigation (login → dashboard)
- Understanding `StatefulWidget` lifecycle: `initState`, `setState`, `dispose`

3.5 Challenges and Solutions

Challenge	Solution
Dashboard loaded before student name was available from <code>SharedPreferences</code>	Used <code>initState</code> to call <code>_loadStudent()</code> which calls <code>setState()</code> once data arrives, triggering a UI rebuild
Dark mode toggle didn't persist after restart	Saved the boolean to <code>SharedPreferences</code> and read it in <code>main.dart</code> using <code>FutureBuilder</code> before the widget tree rendered
Back button on Dashboard exited the app unexpectedly	Set <code>automaticallyImplyLeading: false</code> on the Dashboard <code>AppBar</code>

3.6 Personal Reflection

Designing multiple connected screens revealed how important it is to plan navigation routes before writing code. Early on I had screens navigating to each other in inconsistent ways, causing confusion about which page was “on top” of the navigation stack. After drawing a simple navigation diagram on paper, the routing became clear and the implementation went smoothly. I also found reusable widget methods essential — without `_buildQuickAction()`, the dashboard code would have been 300+ lines of repeated container code.

Week 4 — Event Handling and Data Management

4.1 Overview

This week introduced SQLite — a full relational database — into the app. Notes and Timetable modules were built with complete Create, Read, Update, Delete (CRUD) operations, real-time search, and a form-validation-based add/edit experience.

4.2 Technologies Introduced

Technology	Package	Purpose
SQLite	sqlite: ^2.3.2	On-device relational database
path	path: ^1.9.0	Locating the database file path
TextEditingController	Flutter built-in	Reading and clearing text field values
showTimePicker	Flutter built-in	Time selection for timetable entries

4.3 Activities Undertaken

Database Helper (database_helper.dart): Created a static DatabaseHelper class that manages a single database connection:

```
class DatabaseHelper {
  static Database? _db;

  static Future<Database> get database async {
    _db ??= await _initDB();
    return _db!;
  }

  static Future<void> _createTables(Database db, int version) async {
    await db.execute('''
      CREATE TABLE IF NOT EXISTS notes (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        content TEXT NOT NULL,
        createdAt TEXT NOT NULL
      )
    ''');
    await db.execute('''
      CREATE TABLE IF NOT EXISTS timetable (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        day TEXT NOT NULL,
        unitCode TEXT NOT NULL,
        unitName TEXT NOT NULL,
        startTime TEXT NOT NULL,
        endTime TEXT NOT NULL,
        room TEXT NOT NULL,
        lecturer TEXT NOT NULL
      )
    ''');
```



```

    )
    ''');
}
}

```

Notes Module (notes_page.dart, add_edit_note_page.dart, note.dart): - Note model class with toMap() and fromMap() factory constructor - Full CRUD: insertNote, getAllNotes, searchNotes, updateNote, deleteNote - Search bar with live filtering using onChanged event - FAB (Floating Action Button) to add new notes - Long-press on a note card reveals edit/delete options via a confirmation dialog

Timetable Module (timetable_page.dart, add_edit_timetable_page.dart, timetable_entry.dart): - TimetableEntry model with 7 fields (day, unitCode, unitName, startTime, endTime, room, lecturer) - Entries ordered by day (Monday → Friday) then by start time using a SQL CASE expression - showTimePicker used for start and end time selection - Search across unit code, unit name, day, and lecturer name simultaneously

Event Handling Applied: - Button taps triggered form validation before writing to SQLite - Search bar onChanged event fetched filtered results from the database in real time - Swipe-to-delete gesture on list tiles with confirmation dialogs

4.4 Skills Gained

- SQLite table creation with PRIMARY KEY, AUTOINCREMENT, and NOT NULL constraints
- Writing CRUD methods using sqflite: insert, query, update, delete, rawQuery
- Using the ??= null-aware assignment operator for singleton database initialisation
- Mapping between Dart model objects and Map<String, dynamic> for SQLite
- Parameterised SQL queries using whereArgs (preventing SQL injection)
- Using TextInputAction to move focus between form fields with the keyboard

4.5 Challenges and Solutions

Challenge	Solution
Timetable entries appeared in random order	Used a rawQuery with ORDER BY CASE day WHEN 'Monday' THEN 0 ... END, startTime ASC
Database not found on device after clearing app data	Moved database path resolution to use getDatabasesPath() from sqflite, which points to the correct persistent location
Edit form was not pre-filling existing data	Passed the existing Note or TimetableEntry object to the add/edit page and populated TextEditingController in initState

4.6 Personal Reflection

Working with SQLite this week made the app feel genuinely useful — data now survives app restarts. The most difficult part was designing the DatabaseHelper as a singleton: I had to learn the ??= operator and understand why database connections should not be opened multiple times. The event handling on forms also became more meaningful here — form submission now writes to a real database rather than just printing to the console, so getting validation right before each database write was essential.

Week 5 — Networking and APIs

5.1 Overview

This week connected the Smartz app to the internet for the first time. A live Campus News Feed screen was built using the DEV.to public REST API, with full JSON parsing, error handling, and an asynchronous loading pattern.

5.2 Technologies Introduced

Technology	Package	Purpose
HTTP	http: ^1.2.1	Making HTTP GET requests
dart:convert	Built-in Dart library	Decoding JSON responses
async / await	Dart language feature	Non-blocking asynchronous code
SocketException	dart:io	Detecting no-internet errors
HttpException	dart:io	Detecting HTTP server errors

5.3 Activities Undertaken

API Service (`api_service.dart`):

```
class ApiService {
  static const String _baseUrl =
    'https://dev.to/api/articles?per_page=30&tag=mobile';

  static Future<List<Post>> fetchPosts() async {
    try {
      final response = await http.get(Uri.parse(_baseUrl))
        .timeout(const Duration(seconds: 10));
      if (response.statusCode == 200) {
        final List<dynamic> data = jsonDecode(response.body);
        return data.map((json) => Post.fromJson(json)).toList();
      } else {
        throw HttpException('Server error: ${response.statusCode}');
      }
    } on SocketException {
      throw Exception('No internet connection. Please check your network.');
```

Post Model (`post.dart`): - Used defensive null-handling in `Post.fromJson()`:
dart title: json['title']?.toString() ?? '', authorName: (json['user'] as Map<String, dynamic>?)['name']?.toString() ?? 'Unknown', tags: (json['tag_list'] as List<dynamic>?)?.map((t) => t?.toString() ?? '').where((t) => t.isNotEmpty).toList() ?? [],

News Feed Screen (news_feed_page.dart): - initState calls `_loadPosts()` which sets `_isLoading = true`, fetches data, then calls `setState` - Loading state: `CircularProgressIndicator` shown centrally - Success state: `ListView.builder` renders each article as a card (title, author, tags, reading time) - Error state: error message + “Retry” button that calls `_loadPosts()` again - Pull-to-refresh using `RefreshIndicator` wrapping the `ListView` - Real-time search filters displayed articles client-side using `onChanged`

Four Error Types Handled:

Error Type	Cause	User Message
SocketException	Device offline / no internet	“No internet connection. Please check your network.”
HttpException	Server returned error code	“Server error: [status code]”
FormatException	Malformed JSON body	“Invalid response from server.”
TimeoutException	Request took > 10 seconds	“Request timed out. Please try again.”

5.4 Skills Gained

- Making asynchronous HTTP GET requests using the `http` package
- Parsing a JSON array response into a list of Dart model objects
- Using `try/catch` blocks to distinguish between different network failure types
- Implementing pull-to-refresh with `RefreshIndicator`
- Understanding the request-response lifecycle and HTTP status codes
- Writing null-safe JSON parsing with the `?.` and `??` operators

5.5 Challenges and Solutions

Challenge	Solution
App crashed on first load when <code>_posts</code> was null during list render	Initialised <code>_posts</code> as an empty list <code>[]</code> and only called <code>setState</code> after data arrived
JSON parsing failed for articles with missing user field	Added null-safe casting: <code>(json['user'] as Map<String, dynamic>?)?.['name']</code>
No feedback shown if the fetch was slow	Added a 10-second timeout using <code>.timeout(const Duration(seconds: 10))</code>
Search filtered incorrectly on non-Latin article titles	Used <code>toLowerCase()</code> on both the query and the article title before comparing

5.6 Personal Reflection

Integrating a REST API brought the app to life in a way that local storage could not. Seeing live articles from the internet appear on the screen was satisfying. The most important lesson this week was defensive programming: the real world internet returns unexpected data, delayed responses, and connection errors. Writing only the happy path breaks real apps. By handling all four error types, the app never crashes — it always shows the user a meaningful message. This week also reinforced the value of separating concerns: `ApiService` handles the network, `Post` handles the data model, and `NewsFeedPage` only handles display.

Week 6 — CAT 1: Integrated Application Assessment

6.1 Overview

Week 6 was the CAT 1 (Continuous Assessment Test 1) for BIT4107. The assessment required demonstrating a fully functional mobile application incorporating all concepts covered in Weeks 1–5. The Smartz Student Portal was extended with a Reports screen that aggregated data from all sources, and a Technical Report document was produced.

6.2 CAT 1 Requirements and Implementation

Requirement	Implementation in Smartz App
User Interface	Material Design with consistent blue theme, dark mode
Navigation between screens	Navigator.push/pushReplacement, 9-screen app
Event handling	Form validation, button taps, search onChange, pull-to-refresh
Local storage (SharedPreferences)	Student profile, session, theme, profile image
Local database (SQLite)	Notes and Timetable CRUD with search
Data retrieval	getAllNotes, getAllTimetableEntries, searchNotes, searchTimetable
Networking / API	DEV.to API, HTTP GET, JSON parsing
Error handling	4 error types handled, Retry button
Reports screen	Aggregated view of all data sources

6.3 Reports Screen Development

Added `database_helper.dart` count query methods:

```
static Future<int> getNotesCount() async {
  final db = await database;
  final result = await db.rawQuery('SELECT COUNT(*) as count FROM notes');
  return (result.first['count'] as int?) ?? 0;
}

static Future<Map<String, int>> getTimetableCountByDay() async {
  final db = await database;
  final maps = await db.rawQuery(
    'SELECT day, COUNT(*) as count FROM timetable GROUP BY day',
  );
  return {for (final m in maps) m['day'] as String: (m['count'] as int?) ?? 0};
}
```

The Reports screen (`reports_page.dart`) displayed: - Student profile card (name, email, reg number, course — from `SharedPreferences`) - Summary statistics: total notes count, total timetable classes, total attendance records - Three most recent notes (from `SQLite` using `getRecentNotes(3)`) - Timetable breakdown by day (Monday–Friday class count per day using `GROUP BY`) - System info panel: storage types used, API endpoint, error handling methods - Pull-to-refresh support using `RefreshIndicator`

6.4 Activities Undertaken

- Wrote SQL `COUNT(*)` queries for notes and timetable
- Wrote `GROUP BY` day query returning `Map<String, int>` for timetable breakdown
- Built the Reports screen combining async calls to `SharedPreferences` and `SQLite`
- Used `Future.wait` pattern to load all data sources concurrently before rendering
- Added `AppBar` refresh button alongside pull-to-refresh
- Wrote the Technical Report document covering all features implemented

6.5 Skills Gained

- SQL aggregate functions: `COUNT(*)`, `GROUP BY`
- Loading data from multiple async sources before rendering a screen
- Writing a technical report documenting architecture, features, and weekly progression
- Demonstrating all course outcomes in a single integrated application

6.6 Challenges and Solutions

Challenge	Solution
Reports page flickered while loading	Showed <code>CircularProgressIndicator</code> while <code>_isLoading</code> was true; only rendered data cards once all futures resolved
<code>GROUP BY</code> query returned empty map when no data existed	Defaulted to <code>count > 0 ? '\$count class...': 'No classes'</code> in the UI layer

6.7 Personal Reflection

The CAT 1 assessment validated the cumulative work of five weeks. Bringing together `SharedPreferences`, `SQLite`, and the API into a single Reports screen showed how the app's modules are interconnected. Writing the Technical Report was also valuable — it forced me to articulate *why* I chose each storage method and how error handling was implemented, deepening my understanding beyond just making things work.

Week 7 — Advanced Database Design and Security

7.1 Overview

Week 7 focused on two critical professional requirements: password security (replacing plaintext storage with SHA-256 hashing) and advanced database design (adding a third SQLite table with a schema designed to Third Normal Form). A complete Attendance Management module was also built.

7.2 Technologies Introduced

Technology	Package	Purpose
crypto	crypto: ^3.0.3	SHA-256 cryptographic hashing
dart:convert	Built-in	UTF-8 byte encoding before hashing
SQLite v3 migration	sqlite onUpgrade	Adding the attendance table without data loss

7.3 Password Security: SHA-256 Hashing

Why plaintext passwords are dangerous: If the device is accessed or the SharedPreferences file is read, plaintext passwords expose user credentials directly. SHA-256 hashing is a one-way transformation — the original password cannot be derived from the stored hash.

Implementation in local_storage.dart:

```
import 'dart:convert';
import 'package:crypto/crypto.dart';

static String hashPassword(String password) {
  final bytes = utf8.encode(password);
  final digest = sha256.convert(bytes);
  return digest.toString();
}
```

Example: "student123" →
"ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f"

Migration for existing users: A password_hashed boolean flag in SharedPreferences tracked whether a user's password had been upgraded:

```
static Future<bool> login(String email, String password) async {
  final prefs = await SharedPreferences.getInstance();
  final savedPassword = prefs.getString('student_password');
  final isHashed = prefs.getBool('password_hashed') ?? false;

  if (isHashed) {
    return savedPassword == hashPassword(password);
  } else {
    // Migrate plaintext → hash on first successful login
    if (savedPassword == password) {
      await prefs.setString('student_password', hashPassword(password));
      await prefs.setBool('password_hashed', true);
      return true;
    }
  }
}
```

```

    return false;
  }
}

```

7.4 Database Design: Third Table (Attendance)

Schema (attendance table):

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique record identifier
studentName	TEXT	NOT NULL	Student's full name
regNumber	TEXT	NOT NULL	Registration number
unitName	TEXT	NOT NULL	Unit attended or missed
date	TEXT	NOT NULL	Date in YYYY-MM-DD format
status	TEXT	NOT NULL	Present, Absent, or Late

Normalisation to 3NF: - **1NF:** All columns hold atomic values; no repeating groups; every row uniquely identified by `id` - **2NF:** All non-key attributes depend on the full primary key (single-column PK, so trivially satisfied) - **3NF:** No transitive dependencies — `studentName` and `regNumber` are direct attributes of the attendance record, not derived from each other

Database version upgrade (onUpgrade handler):

```

return openDatabase(
  path,
  version: 3,
  onCreate: _createTables,
  onUpgrade: (db, oldVersion, newVersion) async {
    await _createTables(db, newVersion);
  },
);

```

Bumping the version number from 2 to 3 triggered `onUpgrade` automatically on existing installations, adding the attendance table without destroying notes or timetable data.

7.5 Attendance Module

Attendance CRUD methods added to DatabaseHelper: - `insertAttendance(Attendance record)` — saves a new record - `getAllAttendance()` — ordered by date DESC - `searchAttendance(query)` — searches unit name, date, and status - `deleteAttendance(int id)` — removes a record - `getAttendanceSummary()` — `SELECT status, COUNT(*) GROUP BY status` → `Map<String, int>` - `getAttendanceCount()` — total number of records

Attendance Page (attendance_page.dart): - Summary bar at the top showing Present / Absent / Late / Total counts in real time - `ListView.builder` listing all attendance records as cards with colour-coded status icons - Real-time search by unit name, date, or status using `searchAttendance()` - Delete with confirmation dialog (prevents accidental removal) - FAB navigates to the Add Attendance form

Add Attendance Page (add_attendance_page.dart): - Student name and registration number pre-filled from SharedPreferences (read-only) - Unit name text field - `showDatePicker` for selecting the attendance date (restricted to past dates) - Status dropdown: Present, Absent, Late — each with its own icon and colour

7.6 Skills Gained

- SHA-256 cryptographic hashing using the `crypto` package
- Backward-compatible database migration using `onUpgrade`
- Designing a relational schema to 3NF
- Writing aggregate GROUP BY queries returning a Dart Map
- Auto-populating form fields from stored session data

7.7 Challenges and Solutions

Challenge	Solution
Existing users with plaintext passwords were locked out after adding hashing	Implemented the migration flag (<code>password_hashed</code>): if false, compare directly; on success, hash and set flag to true
SQLite <code>onUpgrade</code> added the new table but existing data was lost on some devices	Changed <code>_createTables</code> to use <code>CREATE TABLE IF NOT EXISTS</code> so it is safe to call on both <code>onCreate</code> and <code>onUpgrade</code>
Attendance summary returned an empty map when no records existed	Handled in UI: checked <code>_summary['Present'] ?? 0</code> with null-aware default

7.8 Personal Reflection

Password hashing was the most professionally significant change made to the app. Before this week, anyone with access to the device's app data could read passwords directly from SharedPreferences. The `crypto` package made SHA-256 hashing straightforward in Dart, but the real challenge was the migration — I had to ensure that users who registered before the update could still log in. The `password_hashed` flag approach was elegant: it required no user action and transparently upgraded their security on the next login. The attendance feature also reinforced proper database design — normalising to 3NF and handling version migration are skills that apply to any professional project.

Week 8 — Handling User Input and Touch Gestures

8.1 Overview

Week 8 focused on implementing touch gesture handling and keyboard input using a class-based Object-Oriented Programming approach. Dedicated controller classes were created to separate event logic from the UI, demonstrating key OOP principles including encapsulation, single responsibility, and dependency injection.

8.2 OOP Class Architecture

Instead of handling all events inside one widget, the week's work was organised into four dedicated controller classes, each responsible for one concern:

```
lib/
├── controllers/
│   ├── event_logger.dart      EventLogger class
│   ├── gesture_controller.dart GestureController class
│   ├── keyboard_controller.dart KeyboardController class
│   └── input_validator.dart   InputValidator class
```

8.3 Controller Classes Implemented

EventLogger (event_logger.dart): Maintains a timestamped in-memory log of all user interactions:

```
class EventLogger {
  final List<String> _events = [];

  void log(String event) {
    final t = DateTime.now();
    final time = '${t.hour.toString().padLeft(2, '0')}: '
                '${t.minute.toString().padLeft(2, '0')}: '
                '${t.second.toString().padLeft(2, '0')}';
    _events.insert(0, '[$time] $event');
    if (_events.length > 30) _events.removeLast();
  }

  List<String> get events => List.unmodifiable(_events);
  void clear() => _events.clear();
}
```

GestureController (gesture_controller.dart): Handles all touch gesture events; depends on EventLogger via constructor injection:

```
class GestureController {
  final EventLogger logger;
  GestureController(this.logger);

  void onTap() => logger.log('☑ Tap detected');
  void onDoubleTap() => logger.log('☑☑ Double tap detected');
  void onLongPress() => logger.log('☑ Long press – context menu displayed');
  void onSwipeLeft() => logger.log('☑ Swipe Left – Previous Page');
  void onSwipeRight() => logger.log('☑ Swipe Right – Next Page');
  void onSwipeUp() => logger.log('☑ Swipe Up – Scroll to top');
```

```
void onSwipeDown() => logger.log('👉 Swipe Down – Refresh');
}
```

KeyboardController (keyboard_controller.dart): Handles text input and keyboard focus events:

```
class KeyboardController {
  final EventLogger logger;
  KeyboardController(this.logger);

  void onTextChanged(String text) => logger.log('👉 Input changed: "$text"');
  void onKeySubmit(String text) => logger.log('👉 Form submitted: "$text"');
  void onFocusGained() => logger.log('👉 Keyboard focus gained');
  void onFocusLost() => logger.log('👉 Keyboard focus lost');
}
```

InputValidator (input_validator.dart): Pure validation logic with no dependencies:

```
class InputValidator {
  String? validateUsername(String? value) {
    if (value == null || value.trim().isEmpty) return 'Username is required';
    if (value.trim().length < 3) return 'Username too short (min 3 characters)';
    if (value.contains(' ')) return 'Username cannot contain spaces';
    return null;
  }

  String getValidationResult(String username) {
    return validateUsername(username) ?? 'Valid username';
  }
}
```

8.4 Input and Gestures Demo Page

The GesturesPage demonstrated all four controller classes working together in one interactive screen.

Gesture Zone: Used Flutter's GestureDetector widget to capture all gesture types:

```
GestDetector(
  onTap: () => _gestureCtrl.onTap(),
  onDoubleTap: () => _gestureCtrl.onDoubleTap(),
  onLongPress: _showLongPressMenu,
  onPanEnd: _handleSwipe, // detects direction from velocity
  child: AnimatedContainer(...), // colour changes with each gesture
)
```

Swipe direction was determined from the drag velocity vector:

```
void _handleSwipe(DragEndDetails details) {
  final dx = details.velocity.pixelsPerSecond.dx;
  final dy = details.velocity.pixelsPerSecond.dy;
  if (dx.abs() > dy.abs()) {
    dx < -200 ? _gestureCtrl.onSwipeLeft() : _gestureCtrl.onSwipeRight();
  } else {
    dy < -200 ? _gestureCtrl.onSwipeUp() : _gestureCtrl.onSwipeDown();
  }
}
```

Long Press — Context Menu: Long press opened a `ModalBottomSheet` with Share, Copy, Delete, and Info options. Selecting any option logged the action via `EventLogger`.

Keyboard Input Section: - `TextFormField` with `FocusNode` attached - Focus gain/loss logged via `KeyboardController` - Each keystroke logged via `onChanged` - `TextInputAction.done` triggered form submission on the keyboard's Enter key - Live inline validation feedback shown below the field in green (valid) or red (invalid) using `InputValidator`

Event Log Section: - Scrollable monospace list displaying the `EventLogger`'s event history - Most recent event shown in bold blue; older entries in grey - `AppBar` trash button clears the log

8.5 OOP Principles Demonstrated

Principle	Where Applied
Encapsulation	Each controller class hides its internal list and implementation
Single Responsibility	Each class does exactly one thing: log, gesture, keyboard, validate
Dependency Injection	<code>GestureController</code> and <code>KeyboardController</code> receive <code>EventLogger</code> via constructor — not hardcoded
Separation of Concerns	UI (<code>GesturesPage</code>) is separate from all logic (controller classes)
Reusability	<code>InputValidator</code> can be used in any form across the app without modification

8.6 Skills Gained

- Using `GestureDetector` for tap, double tap, long press, and pan (swipe) gestures
- Detecting swipe direction from `DragEndDetails.velocity.pixelsPerSecond`
- Using `FocusNode` to respond to keyboard focus gain and loss events
- Triggering form submission via `TextInputAction.done` (keyboard Enter key)
- Designing Dart classes with constructor dependency injection
- Applying OOP principles (encapsulation, SRP, DI) to organise event-driven code

8.7 Challenges and Solutions

Challenge	Solution
<code>onPanEnd</code> triggered for small unintentional movements	Added a minimum velocity threshold of 200 pixels/second before registering a swipe
Double tap and long press conflicted with single tap recognition	Flutter's <code>GestureDetector</code> handles gesture disambiguation automatically; no manual delay was needed
Validation feedback showed below the field only after form submission	Used <code>onChanged</code> + <code>setState</code> to trigger live validation on every keystroke, updating the feedback container in real time
<code>FocusNode</code> listener was not disposed, causing a memory leak warning	Added <code>_focusNode.dispose()</code> in the widget's <code>dispose()</code> method

8.8 Personal Reflection

This week fundamentally changed how I think about structuring code. Before, all logic lived inside the widget's `State` class, making it difficult to test or reuse. Creating four separate controller classes — each with a single, clear responsibility — made the code cleaner and easier to reason about. The dependency injection pattern (passing `EventLogger` into `GestureController` and `KeyboardController` via their constructors) meant that all three

classes shared one log instance without any global variable or singleton. If I ever need to replace `EventLogger` with a cloud-based logger, only the `GesturesPage` needs to change — the controllers themselves remain untouched. This is the kind of design thinking that separates maintainable professional code from code that works today but becomes difficult tomorrow.

Final Application Summary

The Smartz Student Portal, developed over eight weeks, is a complete multi-screen Android application built with Flutter and Dart.

Application Screens

Screen	Description
Welcome	Entry point with Login and Register navigation

Screen	Description
Login	Email + password validation; session management
Register	Student registration form; data saved to SharedPreferences
Dashboard	10 quick-action cards; student profile header
My Profile	Student details; profile image picker
My Notes	SQLite CRUD + real-time search
My Timetable	SQLite CRUD + search; grouped and ordered by day
Campus News Feed	DEV.to REST API; JSON parsing; error handling; search
Attendance	SQLite CRUD + summary bar + search
Reports	Aggregated data from all sources
Input & Gestures	Touch gesture demo; keyboard input; OOP controllers
Settings	Dark mode; change password; profile image; logout
Courses / Assignments / Results / Notifications	Placeholder screens

Technologies Used

Category	Technology
Framework	Flutter 3.x
Language	Dart
Key-value storage	SharedPreferences
Relational database	SQLite (sqflite, v3, 3 tables)
Networking	http package, REST API (DEV.to)
Security	SHA-256 (crypto package)
Image	image_picker
OOP Architecture	Controller classes, dependency injection

Logbook prepared by Makwae Ethan Hope — BIT/2023/72445 BIT4107 Mobile Application Development — Mount Kenya University